Relationship Between Python Files in This Repo

How to read this document

There are n=17 Python files in this repo, so there are n(n-1)/2=136 unordered file pairs. For each pair, we provide one paragraph that explains direct import coupling (if any), the functional relationship in the end-to-end watch-time prediction pipeline, and why that relationship is required for training/evaluation.

Python file inventory

`dataloader/__init__.py`
`dataloader/kuairec.py`
`dataset/kuairec/kuairec_process.py`
`model/__init__.py`
`model/cread.py`
`model/d2q.py`
`model/egmn.py`
`model/layers.py`
`model/tpm.py`
`model/wd.py`
`run_cread.py`
`run_d2co.py`
`run_d2q.py`
`run_egmn.py`
`run_tpm.py`
`run_vr.py`
`utils.py`

Pairwise relationships (one paragraph per unordered pair)

Pair 1/136: dataloader/___init___.py <-> dataloader/kuairec.py **Discussion:** Direct coupling: `dataloader/__init__.py` imports/depends on `dataloader/kuairec.py`. Role fit: `dataloader/__init` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors. Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 2/136: dataloader/___init___.py <-> dataset/kuairec/kuairec_process.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema. Data contract: preprocessing creates the pickled DataFrames plus a description schema; the dataloader enforces that schema at runtime (dtype casting for embeddings, masks for sequences). This relationship is required so models receive correctly typed tensors and consistent feature keys. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 3/136: dataloader/___init___.py <-> model/___init___.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 4/136: dataloader/___init___.py <-> model/cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/cread.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in B: `class Cread`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 5/136: dataloader/___init___.py <-> model/d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/d2q.py` is a PyTorch model definition (feature encoding and forward

computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in B: `class D2Q`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 6/136: dataloader/___init___.py <-> model/egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in B: `class EGMN`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 7/136: dataloader/___init___.py <-> model/layers.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/layers.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in B: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 8/136: dataloader/___init___.py <-> model/tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key

definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 9/136: dataloader/___init___.py <-> model/wd.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 10/136: dataloader/___init___.py <-> run_cread.py Discussion: Direct coupling: `run_cread.py` imports/depends on `dataloader/__init__.py`. Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 11/136: dataloader/___init___.py <-> run_d2co.py Discussion: Direct coupling: `run_d2co.py` imports/depends on `dataloader/__init__.py`. Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)` .... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 12/136: dataloader/___init___.py <-> run_d2q.py **Discussion:** Direct coupling: `run_d2q.py` imports/depends on `dataloader/__init__.py`. Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 13/136: dataloader/___init___.py <-> run_egmn.py **Discussion:** Direct coupling: `run_egmn.py` imports/depends on `dataloader/__init__.py`. Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 14/136: dataloader/___init___.py <-> run_tpm.py **Discussion:** Direct coupling: `run_tpm.py` imports/depends on `dataloader/__init__.py`. Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 15/136: dataloader/___init___.py <-> run_vr.py **Discussion:** Direct coupling: `run_vr.py` imports/depends on `dataloader/__init__.py`. Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`,

`def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 16/136: dataloader/___init___.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/__init__.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `utils.py` is a shared utility/metrics module. Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 17/136: dataloader/kuairec.py <-> dataset/kuairec/kuairec_process.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema. Data contract: preprocessing creates the pickled DataFrames plus a description schema; the dataloader enforces that schema at runtime (dtype casting for embeddings, masks for sequences). This relationship is required so models receive correctly typed tensors and consistent feature keys. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 18/136: dataloader/kuairec.py <-> model/___init___.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor

types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 19/136: dataloader/kuairec.py <-> model/cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/cread.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `class Cread`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 20/136: dataloader/kuairec.py <-> model/d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `class D2Q`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 21/136: dataloader/kuairec.py <-> model/egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `class EGMN`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between

alternatives.

Pair 22/136: dataloader/kuairec.py <-> model/layers.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/layers.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 23/136: dataloader/kuairec.py <-> model/tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 24/136: dataloader/kuairec.py <-> model/wd.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Tensor interface: the dataloader yields features as a dict of tensors whose keys and shapes must match what the model expects for embedding lookups, sequence pooling with masks, and continuous-feature projections. If either side drifts, training breaks or silently degrades. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability

between alternatives.

Pair 25/136: dataloader/kuairec.py <-> run_cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 26/136: dataloader/kuairec.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)` .... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 27/136: dataloader/kuairec.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes

with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 28/136: dataloader/kuairec.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 29/136: dataloader/kuairec.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 30/136: dataloader/kuairec.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 31/136: dataloader/kuairec.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataloader/kuairec.py` is a PyTorch Dataset/DataLoader bridge from preprocessed data to tensors, while `utils.py` is a shared utility/metrics module. Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in A: `class KUAIRECDataset`, `class KUAIRECDataLoader`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 32/136: dataset/kuairec/kuairec_process.py <-> model/___init___.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 33/136: dataset/kuairec/kuairec_process.py <-> model/cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `model/cread.py` is a PyTorch model definition (feature encoding and forward computation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `class Cread`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 34/136: dataset/kuairec/kuairec_process.py <-> model/d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and train-

ing protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `class D2Q`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 35/136: dataset/kuairec/kuairec_process.py <-> model/egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `class EGMN`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 36/136: dataset/kuairec/kuairec_process.py <-> model/layers.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `model/layers.py` is a PyTorch model definition (feature encoding and forward computation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 37/136: dataset/kuairec/kuairec_process.py <-> model/tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Cross-cutting dependency: the two files

participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 38/136: dataset/kuairec/kuairec_process.py <-> model/wd.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 39/136: dataset/kuairec/kuairec_process.py <-> run_cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Reproducibility path: run scripts assume dataset artifacts exist with specific filenames and auxiliary CSVs (e.g., duration buckets/quantiles). Preprocessing establishes those artifacts so training scripts run deterministically. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 40/136: dataset/kuairec/kuairec_process.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Reproducibility path: run scripts assume dataset artifacts exist with specific filenames and auxiliary CSVs (e.g., duration buckets/quantiles). Preprocessing establishes those artifacts so training scripts run deterministically. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`, ... Why

this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 41/136: dataset/kuairec/kuairec_process.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Reproducibility path: run scripts assume dataset artifacts exist with specific filenames and auxiliary CSVs (e.g., duration buckets/quantiles). Preprocessing establishes those artifacts so training scripts run deterministically. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 42/136: dataset/kuairec/kuairec_process.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Reproducibility path: run scripts assume dataset artifacts exist with specific filenames and auxiliary CSVs (e.g., duration buckets/quantiles). Preprocessing establishes those artifacts so training scripts run deterministically. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 43/136: dataset/kuairec/kuairec_process.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Reproducibility path: run scripts assume dataset artifacts exist with specific filenames and auxiliary CSVs (e.g., duration buckets/quantiles). Preprocessing establishes those artifacts so training scripts run deterministically. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) re-

quires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 44/136: dataset/kuairec/kuairec_process.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Reproducibility path: run scripts assume dataset artifacts exist with specific filenames and auxiliary CSVs (e.g., duration buckets/quantiles). Preprocessing establishes those artifacts so training scripts run deterministically. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 45/136: dataset/kuairec/kuairec_process.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `dataset/kuairec/kuairec_process.py` is a preprocessing script that produces the serialized dataset artifacts and schema, while `utils.py` is a shared utility/metrics module. Cross-cutting dependency: the two files participate in the same watch-time pipeline by sharing the dataset schema conventions, feature keys, and the convention that models consume a feature dict and emit predictions that scripts can evaluate. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label` `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 46/136: model/___init___.py <-> model/cread.py Discussion: Direct coupling: `model/__init__.py` imports/depends on `model/cread.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `model/cread.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in B: `class Cread`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 47/136: model/___init___.py <-> model/d2q.py Discussion: Direct coupling: `model/__init__.py` imports/depends on `model/d2q.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in B: `class D2Q`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 48/136: model/___init___.py <-> model/egmn.py Discussion: Direct coupling: `model/__init__.py` imports/depends on `model/egmn.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in B: `class EGMN`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 49/136: model/___init___.py <-> model/layers.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `model/layers.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in B: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 50/136: model/___init___.py <-> model/tpm.py Discussion: Direct coupling: `model/__init__.py` imports/depends on `model/tpm.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from

the modeling idea, not from inconsistent feature plumbing. Key definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 51/136: model/___init___.py <-> model/wd.py **Discussion:** Direct coupling: `model/__init__.py` imports/depends on `model/wd.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 52/136: model/___init___.py <-> run_cread.py **Discussion:** Direct coupling: `run_cread.py` imports/depends on `model/__init__.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criteri` `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 53/136: model/___init___.py <-> run_d2co.py **Discussion:** Direct coupling: `run_d2co.py` imports/depends on `model/__init__.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 54/136: model/___init___.py <-> run__d2q.py **Discussion:** Direct coupling: `run_d2q.py` imports/depends on `model/__init__.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value` `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 55/136: model/___init___.py <-> run__egmn.py **Discussion:** Direct coupling: `run_egmn.py` imports/depends on `model/__init__.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 56/136: model/___init___.py <-> run__tpm.py **Discussion:** Direct coupling: `run_tpm.py` imports/depends on `model/__init__.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 57/136: model/___init___.py <-> run__vr.py **Discussion:** Direct coupling: `run_vr.py` imports/depends on `model/__init__.py`. Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in B: `def get_args(...)`, `def`

`get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 58/136: model/___init___.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/__init__.py` is a PyTorch model definition (feature encoding and forward computation), while `utils.py` is a shared utility/metrics module. Shared scoring/decoding: even without a direct import, models are evaluated using utils' metrics and (for TPM) rely on utils encoding/decoding helpers to convert structured outputs into an expected watch-time. This ensures the model outputs are measurable and comparable. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)` `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 59/136: model/cread.py <-> model/d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class Cread`. Key definitions in B: `class D2Q`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 60/136: model/cread.py <-> model/egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class Cread`. Key definitions in B: `class EGMN`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports

some part of that chain or the comparability between alternatives.

Pair 61/136: model/cread.py <-> model/layers.py Discussion: Direct coupling: `model/cread.py` imports/depends on `model/layers.py`. Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `model/layers.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class Cread`. Key definitions in B: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 62/136: model/cread.py <-> model/tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class Cread`. Key definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 63/136: model/cread.py <-> model/wd.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class Cread`. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 64/136: model/cread.py <-> run_cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and

shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class Cread`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 65/136: model/cread.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class Cread`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second` .... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 66/136: model/cread.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class Cread`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 67/136: model/cread.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class Cread`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 68/136: model/cread.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class Cread`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 69/136: model/cread.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class Cread`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 70/136: model/cread.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and

shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/cread.py` is a PyTorch model definition (feature encoding and forward computation), while `utils.py` is a shared utility/metrics module. Shared scoring/decoding: even without a direct import, models are evaluated using utils' metrics and (for TPM) rely on utils encoding/decoding helpers to convert structured outputs into an expected watch-time. This ensures the model outputs are measurable and comparable. Key definitions in A: `class Cread`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 71/136: model/d2q.py <-> model/egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class D2Q`. Key definitions in B: `class EGMN`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 72/136: model/d2q.py <-> model/layers.py Discussion: Direct coupling: `model/d2q.py` imports/depends on `model/layers.py`. Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `model/layers.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class D2Q`. Key definitions in B: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 73/136: model/d2q.py <-> model/tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while

`model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class D2Q`. Key definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 74/136: model/d2q.py <-> model/wd.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class D2Q`. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 75/136: model/d2q.py <-> run_cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class D2Q`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 76/136: model/d2q.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script

trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class D2Q`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)` .... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 77/136: model/d2q.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class D2Q`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 78/136: model/d2q.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class D2Q`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 79/136: model/d2q.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the

script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class D2Q`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 80/136: model/d2q.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class D2Q`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 81/136: model/d2q.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/d2q.py` is a PyTorch model definition (feature encoding and forward computation), while `utils.py` is a shared utility/metrics module. Shared scoring/decoding: even without a direct import, models are evaluated using utils' metrics and (for TPM) rely on utils encoding/decoding helpers to convert structured outputs into an expected watch-time. This ensures the model outputs are measurable and comparable. Key definitions in A: `class D2Q`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 82/136: model/egmn.py <-> model/layers.py Discussion: Direct coupling: `model/egmn.py` imports/depends on `model/layers.py`. Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `model/layers.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class EGMN`. Key definitions in B: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class`

`FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 83/136: model/egmn.py <-> model/tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class EGMN`. Key definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 84/136: model/egmn.py <-> model/wd.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class EGMN`. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 85/136: model/egmn.py <-> run_cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class EGMN`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and

artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 86/136: model/egmn.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class EGMN`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)` .... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 87/136: model/egmn.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class EGMN`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 88/136: model/egmn.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class EGMN`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing

produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 89/136: model/egmn.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class EGMN`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 90/136: model/egmn.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class EGMN`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 91/136: model/egmn.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/egmn.py` is a PyTorch model definition (feature encoding and forward computation), while `utils.py` is a shared utility/metrics module. Shared scoring/decoding: even without a direct import, models are evaluated using utils' metrics and (for TPM) rely on utils encoding/decoding helpers to convert structured outputs into an expected watch-time. This ensures the model outputs are measurable and comparable. Key definitions in A: `class EGMN`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a cho-

sen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 92/136: model/layers.py <-> model/tpm.py **Discussion:** Direct coupling: `model/tpm.py` imports/depends on `model/layers.py`. Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPercepti` `class DurationMultiLayerPerceptron`, ... Key definitions in B: `class TPM`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 93/136: model/layers.py <-> model/wd.py **Discussion:** Direct coupling: `model/wd.py` imports/depends on `model/layers.py`. Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 94/136: model/layers.py <-> run_cread.py **Discussion:** Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between

alternatives.

Pair 95/136: model/layers.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 96/136: model/layers.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 97/136: model/layers.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`,

`class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 98/136: model/layers.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 99/136: model/layers.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPerceptron`, `class DurationMultiLayerPerceptron`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 100/136: model/layers.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/layers.py` is a PyTorch model definition (feature encoding and forward computation), while `utils.py` is a shared utility/metrics module. Shared scoring/decoding: even without a direct im-

port, models are evaluated using utils' metrics and (for TPM) rely on utils encoding/decoding helpers to convert structured outputs into an expected watch-time. This ensures the model outputs are measurable and comparable. Key definitions in A: `class FeaturesLinear`, `class FeaturesEmbedding`, `class SeqFeatureEmbedding`, `class FactorizationMachine`, `class MultiLayerPercepti` `class DurationMultiLayerPerceptron`, ... Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)` `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 101/136: model/tpm.py <-> model/wd.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `model/wd.py` is a PyTorch model definition (feature encoding and forward computation). Baseline comparability: two model files relate by sharing a common input interface and being swappable under similar training code. This is required to make ablation results meaningful: differences should come from the modeling idea, not from inconsistent feature plumbing. Key definitions in A: `class TPM`. Key definitions in B: `class WideAndDeep`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 102/136: model/tpm.py <-> run_cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class TPM`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 103/136: model/tpm.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Training con-

tract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class TPM`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)` .... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 104/136: model/tpm.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class TPM`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 105/136: model/tpm.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class TPM`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 106/136: model/tpm.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Training

contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class TPM`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 107/136: model/tpm.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class TPM`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 108/136: model/tpm.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/tpm.py` is a PyTorch model definition (feature encoding and forward computation), while `utils.py` is a shared utility/metrics module. Shared scoring/decoding: even without a direct import, models are evaluated using utils' metrics and (for TPM) rely on utils encoding/decoding helpers to convert structured outputs into an expected watch-time. This ensures the model outputs are measurable and comparable. Key definitions in A: `class TPM`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 109/136: model/wd.py <-> run_cread.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/wd.py` is a PyTorch model definition (feature encoding and forward computation), while `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribu-

tion parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class WideAndDeep`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criteri` `def get_split_nodes(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 110/136: model/wd.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/wd.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class WideAndDeep`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 111/136: model/wd.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/wd.py` is a PyTorch model definition (feature encoding and forward computation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class WideAndDeep`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value` `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 112/136: model/wd.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/wd.py` is a PyTorch model definition (feature encoding and forward computation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Training

contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class WideAndDeep`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 113/136: model/wd.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/wd.py` is a PyTorch model definition (feature encoding and forward computation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class WideAndDeep`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 114/136: model/wd.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/wd.py` is a PyTorch model definition (feature encoding and forward computation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Training contract: the run script defines how model outputs are interpreted (scalar vs vector vs distribution parameters) and which loss is applied. The model must provide outputs in the exact shape the script trains against (e.g., mixture parameters, threshold probabilities, tree-node probabilities). Key definitions in A: `class WideAndDeep`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 115/136: model/wd.py <-> utils.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `model/wd.py` is a PyTorch model definition (feature encoding and forward computation), while `utils.py` is a shared utility/metrics module. Shared scoring/decoding: even without a direct import, models are evaluated using utils' metrics and (for TPM) rely on utils encoding/decoding helpers to convert structured outputs into an expected watch-time. This ensures the model out-

puts are measurable and comparable. Key definitions in A: `class WideAndDeep`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 116/136: run_cread.py <-> run_d2co.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second` .... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 117/136: run_cread.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(...)`, `def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 118/136: run_cread.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_cread.py` is an experiment driver (argument parsing, training loop, evaluation), while

run_egmn.py is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: def get_args(...), def get_loaders(...), def discretize_time_label(...), def restore_time_label(...), def get_ord_criterion(...), def get_split_nodes(...), ... Key definitions in B: def get_args(...), def get_loaders(...), def mae_rescale_to_second(...), def test(...). Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 119/136: run_cread.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: run_cread.py is an experiment driver (argument parsing, training loop, evaluation), while run_tpm.py is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: def get_args(...), def get_loaders(...), def discretize_time_label(...), def restore_time_label(...), def get_ord_criterion(...), def get_split_nodes(...), ... Key definitions in B: def get_args(...), def get_loaders(...), def mae_rescale_to_second(...), def test(...). Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 120/136: run_cread.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: run_cread.py is an experiment driver (argument parsing, training loop, evaluation), while run_vr.py is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: def get_args(...), def get_loaders(...), def discretize_time_label(...), def restore_time_label(...), def get_ord_criterion(...), def get_split_nodes(...), ... Key definitions in B: def get_args(...), def get_loaders(...), def mae_rescale_to_second(...), def test(...). Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 121/136: run_cread.py <-> utils.py Discussion: Direct coupling: run_cread.py imports/depends on utils.py. Role fit: run_cread.py is an experiment driver (argument parsing, training loop, evaluation), while utils.py is a shared utility/metrics module. Evaluation

contract: the run script collects predictions/labels and delegates metric computation to utils (MAE/XAUC/KL). This keeps evaluation consistent across methods; otherwise comparisons would be unreliable. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def discretize_time_label(`
`def restore_time_label(...)`, `def get_ord_criterion(...)`, `def get_split_nodes(...)`, ...
Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 122/136: run_d2co.py <-> run_d2q.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`, ...
Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 123/136: run_d2co.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`
.... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 124/136: run_d2co.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conven-

tions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)` .... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 125/136: run_d2co.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 126/136: run_d2co.py <-> utils.py Discussion: Direct coupling: `run_d2co.py` imports/depends on `utils.py`. Role fit: `run_d2co.py` is an experiment driver (argument parsing, training loop, evaluation), while `utils.py` is a shared utility/metrics module. Evaluation contract: the run script collects predictions/labels and delegates metric computation to utils (MAE/XAUC/KL). This keeps evaluation consistent across methods; otherwise comparisons would be unreliable. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def get_gmm_mean(...)`, `def get_gmm_label(...)`, `def get_real_value(...)`, `def mae_rescale_to_second(...)`, ... Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 127/136: run_d2q.py <-> run_egmn.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and

shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 128/136: run_d2q.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 129/136: run_d2q.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quantile(...)`, `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 130/136: run_d2q.py <-> utils.py Discussion: Direct coupling: `run_d2q.py` imports/depends on `utils.py`. Role fit: `run_d2q.py` is an experiment driver (argument parsing, training loop, evaluation), while `utils.py` is a shared utility/metrics module. Evaluation contract: the run script collects predictions/labels and delegates metric computation to utils (MAE/XAUC/KL). This keeps evaluation consistent across methods; otherwise comparisons would be unreliable. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def label_norm(...)`, `def from_value_to_quan` `def from_quantile_to_value(...)`, `def get_buckets_infor(...)`, ... Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_loss(...)`, `def get_tree_encoded_la` `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 131/136: run_egmn.py <-> run_tpm.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 132/136: run_egmn.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_egmn.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_secon` `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 133/136: run_egmn.py <-> utils.py Discussion: Direct coupling: `run_egmn.py` imports/depends on `utils.py`. Role fit: `run_egmn.py` is an experiment driver (argument pars-

ing, training loop, evaluation), while `utils.py` is a shared utility/metrics module. Evaluation contract: the run script collects predictions/labels and delegates metric computation to utils (MAE/XAUC/KL). This keeps evaluation consistent across methods; otherwise comparisons would be unreliable. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(` `def test(...)`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_l` `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 134/136: run_tpm.py <-> run_vr.py Discussion: Direct coupling: there is no explicit import edge between the two files, but they still connect through shared conventions and shared runtime objects (feature schema, tensor shapes, metrics, and training protocol). Role fit: `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation), while `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation). Protocol family: two run scripts relate by implementing alternative objectives/label transforms under a common dataset interface. This is required to benchmark multiple methods on the same data with the same metrics. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Key definitions in B: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_secon` `def test(...)`. Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 135/136: run_tpm.py <-> utils.py Discussion: Direct coupling: `run_tpm.py` imports/depends on `utils.py`. Role fit: `run_tpm.py` is an experiment driver (argument parsing, training loop, evaluation), while `utils.py` is a shared utility/metrics module. Evaluation contract: the run script collects predictions/labels and delegates metric computation to utils (MAE/XAUC/KL). This keeps evaluation consistent across methods; otherwise comparisons would be unreliable. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`, `def test(...)`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_l` `def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`, `def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.

Pair 136/136: run_vr.py <-> utils.py Discussion: Direct coupling: `run_vr.py` imports/depends on `utils.py`. Role fit: `run_vr.py` is an experiment driver (argument parsing, training loop, evaluation), while `utils.py` is a shared utility/metrics module. Evaluation contract: the run script collects predictions/labels and delegates metric computation to utils (MAE/XAUC/KL). This keeps evaluation consistent across methods; otherwise comparisons would be unreliable. Key definitions in A: `def get_args(...)`, `def get_loaders(...)`, `def mae_rescale_to_second(...)`,

`def test(...)`. Key definitions in B: `def get_playtime_percentiles_range(...)`, `def get_tree_classify_l`
`def get_tree_encoded_label(...)`, `def get_tree_encoded_value(...)`, `class InversePairsCalc`,
`def eval_xauc(...)`, ... Why this relationship serves the model: the end-to-end objective (predict normalized watch-time) requires a consistent chain: preprocessing produces schema and artifacts; dataloader enforces tensor types/shapes; model computes outputs; run script optimizes with a chosen loss; utils evaluates with shared metrics. Each pair supports some part of that chain or the comparability between alternatives.